A PROJECT REPORT

ON

Yojna Mitra – Simplifying Access to Government Welfare

Submitted in Partial Fulfillment of the Requirements for the Award of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING (2026)

By

Anshuman Singh (2205051530032)

Under the Guidance of

Prof. (Dr.) Arunendra Singh

ALLENHOUSE INSTITUTE OF TECHNOLOGY, KANPUR

Affiliated to

Dr. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW (U.P.)

CERTIFICATE

This is to certify that the project entitled **"Yojna Mitra – Simplifying Access to Government Welfare"**, submitted by **Anshuman Singh (2205051530032)** in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (Computer Science & Engineering)** from **Dr. A.P.J. Abdul Kalam Technical University (Uttar Pradesh, Lucknow)**, is a record of his original work carried out under our supervision and guidance.

The project report embodies the results of original work and studies carried out by the student, and to the best of our knowledge, the contents of this report do not form the basis for the award of any other degree or diploma to the candidate or to any other person.

Prof. (Dr.) Arunendra Singh
Project Guide

Prof. (Dr.) Sudhir Kumar Singh
HOD-CSE

DECLARATION

I hereby declare that the project entitled **"Yojna Mitra – Simplifying Access to Government Welfare"**, submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology (Computer Science & Engineering)** from **Dr. A.P.J. Abdul Kalam Technical University (Uttar Pradesh, Lucknow)**, is a record of my original work carried out under the supervision and guidance of **Prof. (Dr.) Arunendra Singh**.

To the best of my knowledge, this project has not been submitted, either in part or in full, to **Dr. A.P.J. Abdul Kalam Technical University** or any other university or institute for the award of any degree or diploma.

Anshuman Singh
2205051530032

ACKNOWLEDGEMENT

I take this opportunity to express my profound gratitude to all those who have contributed to the successful completion of this project.

I am deeply indebted to my project guide, **Prof. (Dr.) Arunendra Singh**, for his exemplary guidance, constant encouragement, and valuable insights throughout the course of this work. His intellectual support, constructive feedback, and motivation have been instrumental in the successful execution of this project and the preparation of this report.

I would also like to extend my sincere appreciation to the faculty members and the Department of Computer Science & Engineering for providing the necessary facilities, resources, and a conducive environment for carrying out this project.

I am equally grateful to my family and friends for their unwavering support, encouragement, and understanding, which helped me remain focused and motivated throughout the project duration.

ABSTRACT

Government welfare schemes are designed to improve the socio-economic conditions of citizens across sectors such as education, healthcare, employment, and entrepreneurship. However, a major challenge lies in ensuring that these schemes effectively reach the intended beneficiaries. Many individuals remain unaware of relevant schemes due to fragmented information sources, lack of personalization, and the complexity of existing government portals . This creates a gap between scheme availability and accessibility, highlighting the need for an intelligent and user-centric solution.

The Yojna Mitra system addresses this issue by developing a **web-based centralized government schemes portal integrated with a recommendation engine**. The platform consolidates scheme information into a single interface, eliminating the need for users to navigate multiple sources. It provides **personalized recommendations** based on user-specific inputs such as category, interests, and level, thereby improving relevance and reducing manual effort.

The system is implemented using the **MERN stack (MongoDB, Express.js, React.js, Node.js)** for the web application, along with a **FastAPI-based Python microservice** for the recommendation engine. The recommendation model adopts a **content-based filtering approach**, which is particularly suitable in scenarios where labeled data is unavailable. Scheme data is processed by combining textual attributes such as description, benefits, eligibility, category, and level. These features are transformed into numerical representations using **TF-IDF (Term Frequency–Inverse Document Frequency)**, and similarity between user input and scheme data is computed using **cosine similarity**.

Based on similarity scores, the system ranks schemes and returns the most relevant results in real time. This approach enables efficient handling of unstructured textual data and ensures accurate recommendations without requiring complex training processes. The platform also features a user-friendly interface, profile management, favorites functionality, and chatbot assistance to enhance usability and accessibility.

Overall, the proposed system significantly improves scheme discovery by providing an intelligent, scalable, and efficient solution. It demonstrates the practical application of machine learning in public service systems and establishes a strong foundation for future enhancements such as hybrid recommendation models, advanced NLP techniques, and feedback-driven personalization.

TABLE OF CONTENTS

LIST OF FIGURE

LIST OF TABLES

LIST OF ABBREVIATIONS AND SYMBOLS USED

Abbreviation	Full Form
ML	Machine Learning
TF-IDF	Term Frequency – Inverse Document Frequency
DB	Database
JWT	JSON Web Token
MERN	MongoDB, Express.js, React.js, Node.js
HTTP	HyperText Transfer Protocol
NLP	Natural Language Processing

CHAPTER 1: INTRODUCTION

1.1 Background

Government schemes are one of the primary instruments used by governments to promote socio-economic welfare and inclusive growth. In India, both central and state governments launch numerous schemes aimed at improving living standards, providing financial assistance, enhancing employment opportunities, supporting entrepreneurship, and ensuring access to education and healthcare.

Despite the availability of these schemes, a significant gap exists between the schemes offered and the beneficiaries who actually utilize them. This gap is primarily due to **lack of awareness, fragmented information sources, and absence of personalized access mechanisms**. Citizens often struggle to find schemes that match their eligibility due to the overwhelming volume of information available online.

Existing platforms such as government portals provide centralized listings of schemes; however, they are mostly static and lack intelligent filtering mechanisms. Users are required to manually search and interpret eligibility criteria, which is time-consuming and inefficient.

With advancements in Artificial Intelligence and Natural Language Processing, it is now possible to build intelligent systems that can process large volumes of unstructured textual data and provide meaningful insights. Recommendation systems, widely used in domains such as e-commerce and entertainment, can be effectively adapted to public service platforms.

The **Yojna Mitra** project is developed as a **web-based centralized schemes portal** that integrates a recommendation engine to provide personalized scheme suggestions. It leverages machine learning techniques to analyze scheme data and match it with user details, thereby simplifying the process of scheme discovery.

1.2 Need for the System

The need for a system like Yojna Mitra arises from multiple challenges present in the current ecosystem of government scheme dissemination.

One of the primary issues is **information overload**. There are hundreds of schemes available at different administrative levels, making it difficult for users to identify which schemes are relevant to them. This leads to confusion and often results in users ignoring potentially beneficial schemes.

Another significant issue is the **lack of personalization**. Existing portals provide generic information without considering user-specific attributes such as category, occupation, income level, or interests. As a result, users must manually analyze whether they are eligible for a scheme.

Additionally, there is a **low level of awareness**, particularly among rural and less digitally literate populations. Many users are unaware of schemes simply because the information is not presented in an accessible or targeted manner.

The **complexity of navigation** further adds to the problem. Users often need to browse multiple pages, apply filters, and read lengthy descriptions, which can be discouraging.

Therefore, there is a strong need for a system that:

- Centralizes scheme information
- Provides personalized recommendations
- Simplifies user interaction
- Reduces search effort
- Improves accessibility

Yojna Mitra fulfills these requirements by combining a centralized portal with an intelligent recommendation system.

1.3 Problem Definition

The core problem addressed by this project is:

> **"To design and develop a web-based system that can recommend relevant government schemes to users based on their details, using unstructured data without relying on labeled datasets."**

Traditional systems fail to provide personalized recommendations due to several limitations:

- They rely on keyword-based search rather than semantic understanding
- They do not adapt to user profiles
- They require manual filtering and interpretation

Another major challenge is that the available dataset is **unstructured** and lacks labeled information indicating which schemes are suitable for which users. This makes it difficult to apply supervised machine learning techniques.

Therefore, the problem involves:

- Processing unstructured textual data
- Extracting meaningful features
- Matching user input with scheme data
- Ranking schemes based on relevance

The system must also ensure scalability, efficiency, and real-time responsiveness.

1.4 Objective of the Project

The primary objective of the Yojna Mitra project is to design and develop a **web-based centralized government schemes portal** that provides **personalized recommendations** to users based on their details. The system aims to simplify the process of discovering relevant schemes by reducing manual effort and improving accessibility through intelligent filtering.

The project integrates modern web technologies with machine learning techniques to create a platform that is both efficient and user-friendly. By combining a centralized database with a recommendation engine, the system ensures that users receive relevant and up-to-date scheme suggestions in real time.

1.4.1 General Objective

The general objective of the project is:

> **To develop an intelligent system that recommends relevant government schemes by analyzing user details and scheme data.**

This objective focuses on enhancing user experience by minimizing the complexity involved in searching for schemes and ensuring that users are guided toward suitable options.

1.4.2 Specific Objectives

To achieve the overall goal, the following specific objectives are defined:

- To develop a **centralized platform** that aggregates government schemes in one place, eliminating the need to access multiple sources
- To implement a **content-based recommendation system** that utilizes scheme information such as description, benefits, eligibility, category, and level
- To apply **TF-IDF (Term Frequency–Inverse Document Frequency)** to convert textual scheme data into numerical vectors
- To use **cosine similarity** to measure the relevance between user input and scheme data
- To rank schemes based on similarity scores and provide the **top relevant recommendations**
- To integrate the recommendation system with a **MERN stack application**, ensuring seamless interaction between frontend, backend, and database
- To develop a **FastAPI-based machine learning service** for handling recommendation logic independently

- To enable **real-time updates** by fetching data dynamically from MongoDB

1.4.3 Functional Objectives

The system is designed to perform the following functions:

- Accept user details such as category, interests, and level
- Process user input and match it with scheme data
- Generate and display recommended schemes
- Provide detailed information about each scheme, including benefits, required documents, and application process
- Ensure fast and efficient response for user queries

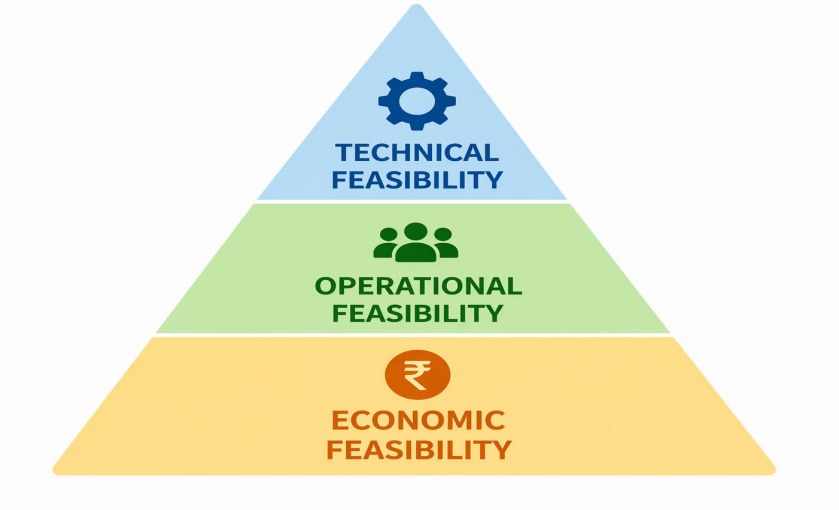
1.4.4 Non-Functional Objectives

The system must also satisfy certain quality requirements:

- **Performance:** Provide quick recommendations with minimal delay
- **Scalability:** Handle increasing numbers of users and schemes efficiently
- **Usability:** Offer a simple and intuitive interface for all users
- **Reliability:** Ensure consistent and accurate recommendations
- **Maintainability:** Allow easy updates and improvements
- **Security:** Protect user data and ensure secure system operations

1.5 Feasibility Study

A feasibility study is conducted to evaluate the practicality and viability of the proposed system before its implementation. It helps determine whether the project can be successfully developed, deployed, and maintained within the available resources and constraints. For the Yojna Mitra project, feasibility is analyzed from three key perspectives: **technical feasibility, operational feasibility, and economic feasibility.**



1.5.1 Technical Feasibility

Technical feasibility assesses whether the required technology, tools, and resources are available to develop and implement the system effectively.

The Yojna Mitra system is built using a modern and widely adopted technology stack, which ensures that the project is technically feasible. The frontend is developed using React.js, providing a dynamic and responsive user interface. The backend is implemented using Node.js and Express.js, which handle API requests and business logic efficiently. MongoDB is used as the database due to its flexibility in handling unstructured data.

The machine learning component is implemented as a separate microservice using FastAPI in Python. This modular architecture allows the ML model to operate independently from the main backend, ensuring better scalability and maintainability.

The recommendation engine uses text-based techniques such as **TF-IDF for feature extraction** and **cosine similarity for ranking schemes**. These techniques are computationally efficient and do not require large-scale training or specialized hardware such as GPUs. This makes the system suitable for real-time applications.

Additionally, integration between components is achieved using REST APIs, ensuring seamless communication between the frontend, backend, and ML service. The use of open-source libraries such as pandas and scikit-learn further simplifies development.

Overall, the availability of required technologies, ease of integration, and efficiency of the chosen algorithms confirm that the project is technically feasible.

1.5.2 Operational Feasibility

Operational feasibility evaluates how well the proposed system will function in a real-world environment and whether it will be accepted by users.

The Yojna Mitra system is designed with a strong focus on usability and accessibility. The web-based interface allows users to access the system from any device with an internet connection, eliminating the need for specialized hardware or software.

The system provides a simple and intuitive interface where users can enter their details and receive recommendations without requiring technical knowledge. This makes it suitable for a wide range of users, including those with limited digital literacy.

The recommendation engine reduces the need for manual searching and filtering, thereby improving efficiency and user satisfaction. The system also presents scheme details in a clear and structured format, making it easier for users to understand eligibility criteria and application processes.

Another important aspect of operational feasibility is adaptability. The system supports real-time updates from the database, ensuring that newly added or modified schemes are immediately reflected in recommendations. This keeps the system relevant and up-to-date.

Furthermore, the modular design of the system allows for easy maintenance and future enhancements. New features can be added without affecting existing functionality.

Thus, the system is operationally feasible as it aligns with user needs, improves usability, and can be effectively deployed in real-world scenarios.

1.5.3 Economic Feasibility

Economic feasibility analyzes the cost-effectiveness of the project and determines whether the benefits outweigh the costs involved in development and deployment.

The Yojna Mitra project is highly cost-effective as it is built using open-source technologies. Tools and frameworks such as React.js, Node.js, MongoDB, FastAPI, and scikit-learn are freely available, eliminating licensing costs.

The system does not require expensive hardware or infrastructure. It can be developed and tested on standard personal computers and deployed on low-cost cloud platforms such as AWS, Render, or Vercel.

The development cost is minimal, as the project primarily involves software development using readily available tools. Maintenance costs are also low due to the modular architecture and use of widely supported technologies.

From a benefits perspective, the system provides significant value by improving access to government schemes, reducing user effort, and increasing awareness. These benefits outweigh the minimal costs involved in development and deployment.

Therefore, the project is economically feasible and sustainable for both academic and real-world applications.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Systems

With the rapid advancement of digital governance, several web-based platforms have been developed to provide centralized access to government schemes and services. These systems aim to improve transparency, accessibility, and efficiency in public service delivery. However, their capabilities vary significantly, particularly in terms of personalization and intelligent recommendation.

2.1.1 myScheme

The myScheme platform is one of the most relevant existing systems, designed as a centralized portal for discovering government schemes.

Key Features:

- Provides a **centralized database of schemes** from multiple ministries and departments
- Allows users to **search schemes using filters** such as category, gender, and eligibility
- Offers a **user-friendly interface** for browsing schemes
- Enables classification of schemes based on beneficiaries (students, farmers, women, etc.)
- Provides **detailed scheme information** including benefits, eligibility, and application process
- Supports **multi-level schemes** (central and state)
- Designed under the Digital India initiative for improved accessibility

Observation:

- Focuses on **information aggregation rather than intelligent recommendation**

2.1.2 UMANG

UMANG is a comprehensive platform providing access to a wide range of government services.

Key Features:

- Integrates **multiple government services into a single platform**
- Provides access to services across **central, state, and local governments**
- Available on **web, mobile apps, SMS, and email**
- Covers domains such as:
    - Education
    - Healthcare
    - Employment
- Offers **user authentication and profile-based access**
- Supports **real-time service delivery**

Observation:

- Focuses on **service execution rather than scheme recommendation.**
- Lacks ML-based personalization

2.1.3 State-Level Government Portals

Various state governments have developed their own portals for delivering schemes and services.

Key Features:

- Provide **state-specific scheme information**
- Offer **online application facilities**
- Enable **status tracking of applications**
- Focus on **regional services and policies**

Observation:

- No intelligent recommendation system

2.1.4 National Government Services Portal

This is the official national portal providing access to government information and services.

Key Features:

- Centralized access to:
    - Schemes
    - Services
    - Policies
- Categorized information for citizens, businesses, and government
- Provides links to multiple departments

Observation:

- Informational portal
- No recommendation system

2.2 Limitations of Existing Systems

Although existing government portals and web-based systems have improved accessibility and centralization of schemes, they still suffer from several limitations that reduce their overall effectiveness.

2.2.1 Lack of Personalization

- Existing systems provide **generic results** to all users
- No customization based on:
    - User category
    - Interests
- Users must manually determine which schemes are relevant
- Leads to **inefficient user experience.**

2.2.2 Manual Search and Filtering

- Users are required to:
    - Enter keywords
    - Apply filters manually
    - Browse multiple pages
- No automated recommendation mechanism
- Increases **time and effort required to find schemes**

2.2.3 Inefficient Search Mechanism

- Based on **keyword matching only**
- Does not understand user intent or context
- Results may be:
    - Irrelevant
    - Incomplete
- Poor semantic matching

2.2.4 Limited Scalability in Some Portals:

- Monolithic system design
- Difficult to scale with increasing data
- Performance degradation with large datasets

2.2.5 Lack of Integration with Intelligent Systems

- No integration with:
    - Machine learning models
    - Recommendation engines
- Systems remain **static rather than adaptive.**

2.3 Related Research Work

A considerable amount of research has been conducted in the domains of **recommendation systems, natural language processing, and intelligent web-based applications**. This section reviews relevant research works that contribute to the development of systems similar to the proposed Yojna Mitra platform.

2.3.1 Government Scheme Recommendation Systems

Several recent studies have focused on improving accessibility to government schemes using intelligent systems.

A research work titled **"GovGenius: Government Schemes Recommendation Web Application" (2024)** proposes a web-based platform designed to recommend schemes based on user profiles. The system emphasizes the importance of centralizing scheme data and providing personalized suggestions. It highlights that lack of awareness and complex navigation are major barriers for citizens. The study demonstrates that integrating recommendation systems with web applications significantly improves accessibility and usability.

Another study, **"AI-Based Government Scheme Recommendation System Using User Profiling and NLP Techniques" (2025)**, introduces an intelligent model that uses demographic attributes such as income, occupation, and region to match users with relevant schemes. The system applies Natural Language Processing techniques to analyze scheme descriptions and improve matching accuracy. The research concludes that AI-driven systems can enhance inclusivity and ensure better targeting of beneficiaries.

2.3.2 Text-Based Recommendation Systems

The study **"Comparative Analysis of Content-Based Filtering and TF-IDF Approaches for Recommendation Systems" (2023)** evaluates different techniques for handling textual data. The results indicate that TF-IDF combined with cosine similarity provides better performance in scenarios where datasets are unstructured and lack user interaction history.

Another research work, **"Scientific Literature Recommendation Using Text Similarity Models" (2020)**, demonstrates how text-based models can be used to recommend relevant documents. The system uses similarity measures to match user queries with document content, reducing the need for manual search.

These studies validate the effectiveness of text-based approaches for recommendation systems similar to Yojna Mitra.

2.3.3 Content-Based Recommendation Using TF-IDF and Cosine Similarity

Content-based filtering has been widely studied in the context of recommendation systems.

The research paper **"A Content-Based Smart Tourist Recommendation System Using TF-IDF and Cosine Similarity" (2022)** demonstrates how textual data can be effectively used to generate recommendations. The system processes user preferences and compares them with item descriptions using TF-IDF and cosine similarity. The results show that the approach provides accurate recommendations with low computational cost.

Similarly, **"Application of TF-IDF and Cosine Similarity in Text-Based Recommendation Systems" (2021)** highlights the effectiveness of these techniques in handling unstructured data. The study confirms that TF-IDF helps identify important features in textual datasets, while cosine similarity provides an efficient way to measure relevance between items.

These approaches are particularly suitable for government scheme data, which is largely textual and lacks structured labels.

2.3.4 Research on Recommendation Systems

The foundational concepts of recommendation systems are discussed in the paper **"Recommender Systems: An Overview" (2019)**. This study categorizes recommendation techniques into:

- Content-based filtering
- Collaborative filtering

- Hybrid systems

The research highlights that content-based filtering is effective when user interaction data is limited, while collaborative filtering performs better in data-rich environments. However, it also identifies challenges such as the **cold-start problem** and **data sparsity**, which limit the effectiveness of collaborative approaches.

2.4 Summary of Literature

· Existing government portals (e.g., myScheme, UMANG) provide **centralized access** to schemes but lack intelligent recommendation features.

· Most systems rely on **manual search and filtering**, resulting in increased user effort and inefficient scheme discovery.

· Current platforms do not utilize **machine learning or AI**, leading to lack of personalization and relevance in results.

· Research indicates that **recommendation systems** are effective in handling large datasets and reducing information overload.

· Among various approaches, **content-based filtering** is most suitable for:

- Unstructured textual data
- Absence of labeled datasets
- Real-time recommendation needs

· Techniques such as **TF-IDF** and **cosine similarity** are widely used and proven effective for text-based recommendation systems.

· Collaborative and hybrid approaches provide higher accuracy but require **large datasets and user interaction history**, making them less suitable for this problem.

· Modern web technologies like **MERN stack** enable scalable, efficient, and user-friendly web applications.

CHAPTER 3: PROPOSED METHODOLOGY

3.1 Problem Formulation

The problem addressed in this project is the **difficulty faced by users in identifying relevant government schemes** from a large collection of available schemes. Existing platforms provide scheme information but require users to manually search, filter, and interpret eligibility criteria.Therefore the objective is to design and develop a **web-based centralized platform** that recommends relevant government schemes to users based on their details using a **content-based machine learning approach**, without relying on labeled datasets.

Input

- User details:
    - Category (e.g., OBC, General, SC/ST)
    - Interests (e.g., education, employment)
    - Level (State / Central)

Output

- Top N recommended schemes including:
    - Title
    - Description
    - Benefits
    - Documents
    - Application process

Key Challenges

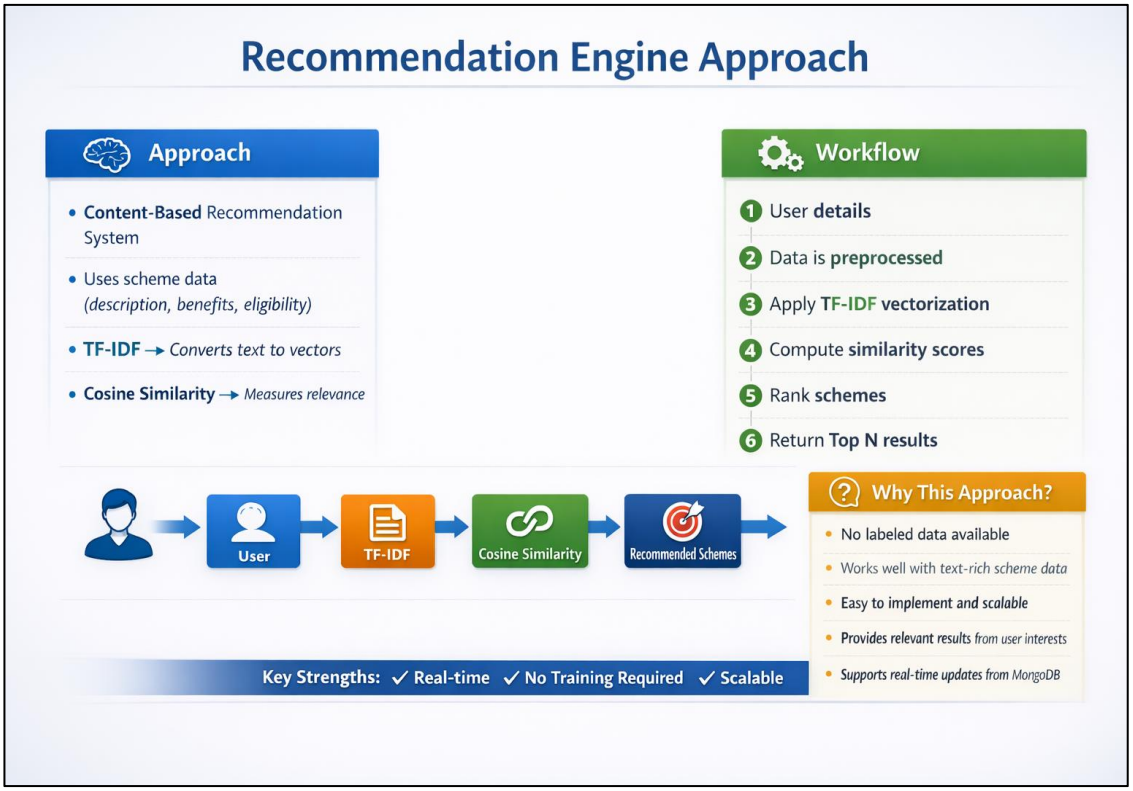
- No labeled dataset (no “relevant / not relevant” tags)
- Data is text-heavy and unstructured
- Need for real-time recommendation
- High number of schemes → information overload

Nature of Data

- Dataset source: **Open Government Data (OGD) Platform India**
- Data type: **Unstructured textual data**
- Fields include:
    - Title
    - Description
    - Benefits / Key Features
    - Eligibility

Proposed Solution

- Use **content-based recommendation system**
- Apply:
    - TF-IDF for feature extraction
    - Cosine similarity for ranking



3.2 System Architecture

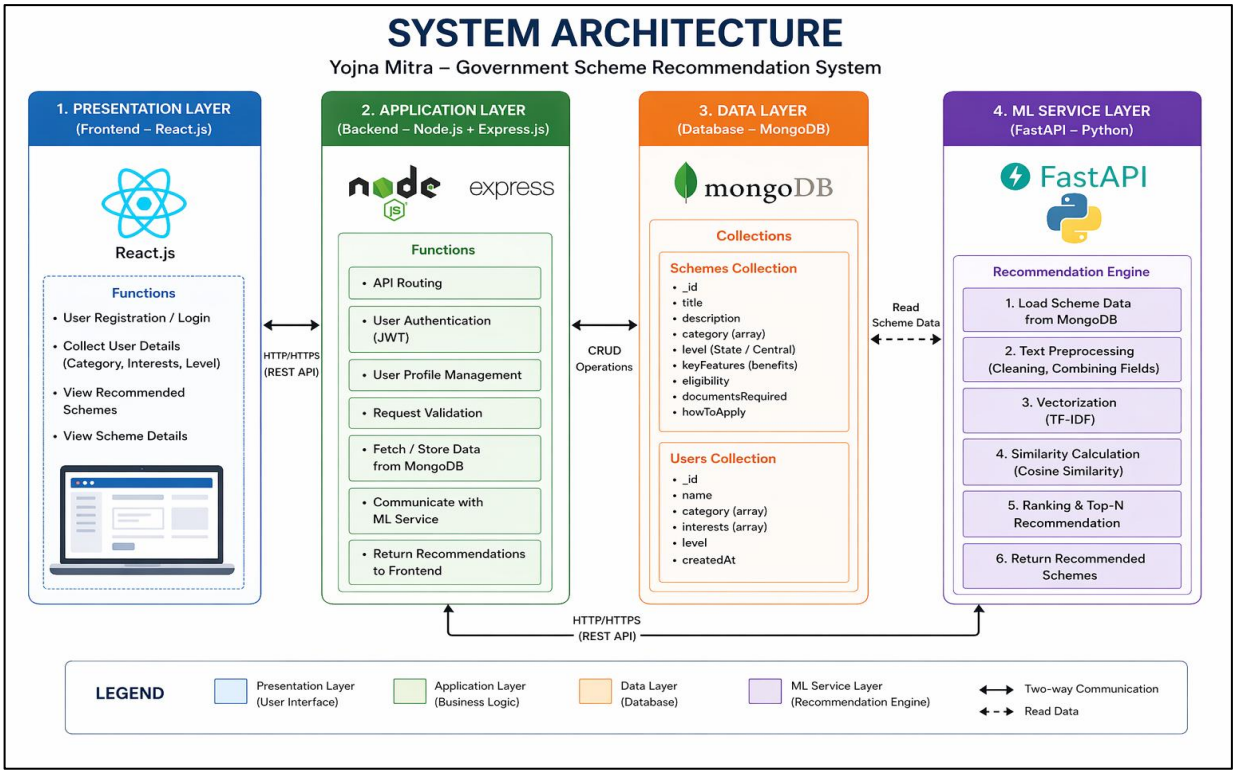
3.2.1 Overview

The proposed system follows a **modular, service-oriented architecture** that integrates a **MERN-based web application** with a **machine learning microservice**. The architecture is designed to ensure:

- Separation of concerns (UI, business logic, ML processing)
- Scalability and maintainability
- Real-time recommendation generation

The system consists of four primary layers:

1. **Presentation Layer (Frontend)**
2. **Application Layer (Backend API)**
3. **Data Layer (Database)**
4. **Machine Learning Service Layer**



3.2.1 Architecture Components

1. Presentation Layer (Frontend – React.js)

The frontend is responsible for user interaction and visualization.

Functions:

- Collects user inputs:
  - Category
  - Interests
  - Level
- Sends API requests to backend
- Displays recommended schemes

Characteristics:

- Component-based UI
- Responsive design
- Uses Axios/Fetch for API communication

2. Application Layer (Backend – Node.js + Express.js)

The backend acts as a **central controller** for the system.

Functions:

- Handles HTTP requests from frontend
- Manages authentication and user profiles
- Fetches user data from MongoDB
- Calls ML service API for recommendations
- Returns processed results to frontend

Key Role:

- Acts as a bridge between frontend and ML service

3. Data Layer (Database – MongoDB)

MongoDB is used to store both **scheme data and user data**.

Data Stored:

Scheme Collection (from OGD platform)

User Collection

Why MongoDB?

- Handles **unstructured and semi-structured data**
- Flexible schema
- Efficient for JSON-based data

4. Machine Learning Service (FastAPI – Python)

This is a **separate microservice** responsible for generating recommendations.

Functions:

- Loads scheme data from MongoDB
- Preprocesses textual data
- Converts text into vectors using TF-IDF
- Computes similarity using cosine similarity
- Returns top recommended schemes

Key Features:

- Stateless API (/recommend)
- Fast response using precomputed vectors
- Supports dynamic updates via caching mechanism

3.2.2 Security Considerations

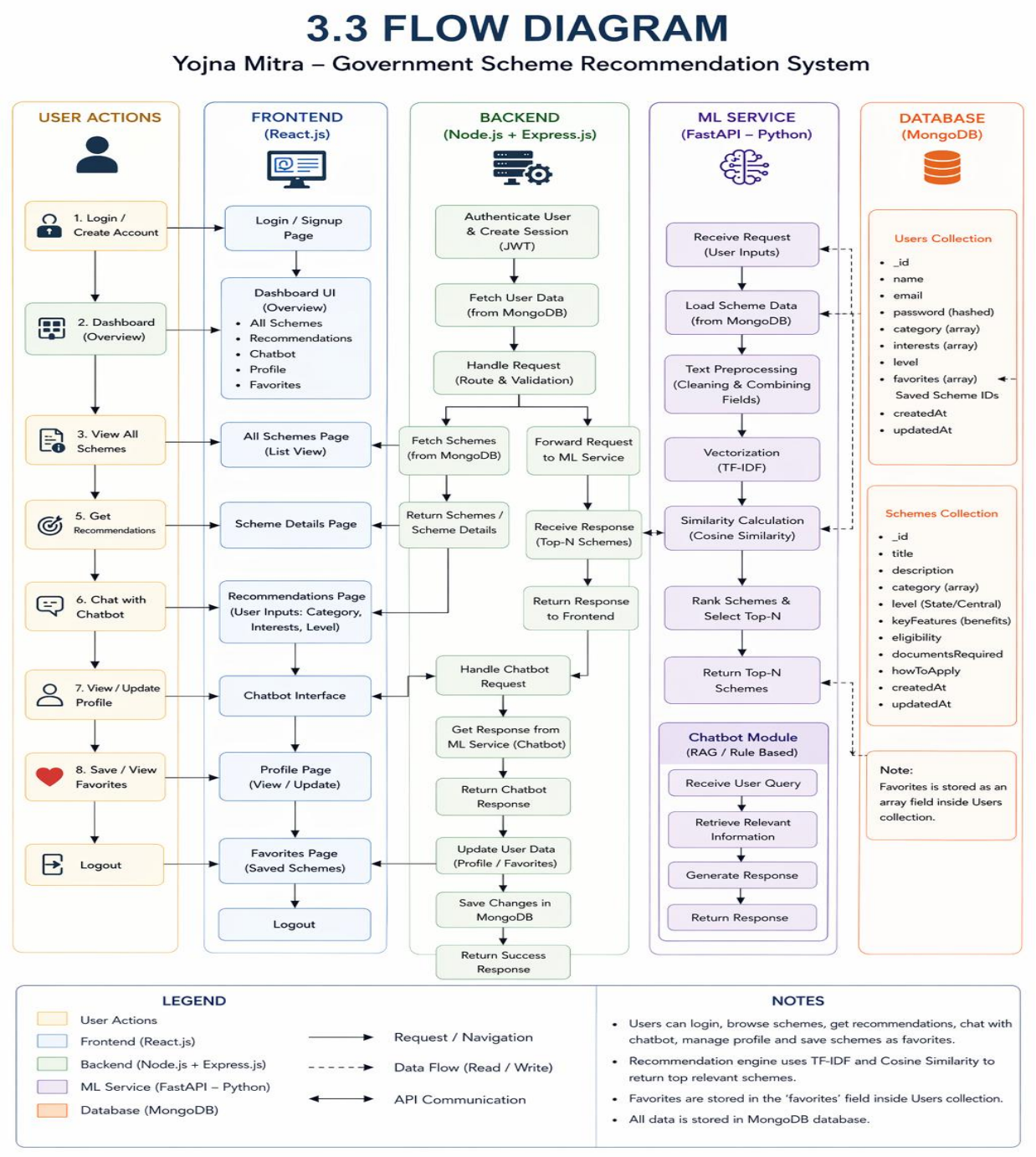
· API validation for user inputs

· Secure database connection (MongoDB Atlas)

· Authentication handled at backend

3.3 Flow Diagram

The flow diagram represents the **complete user journey and system workflow** in the Yojna Mitra platform. It includes user authentication, scheme browsing, recommendation generation, chatbot interaction, and profile management.



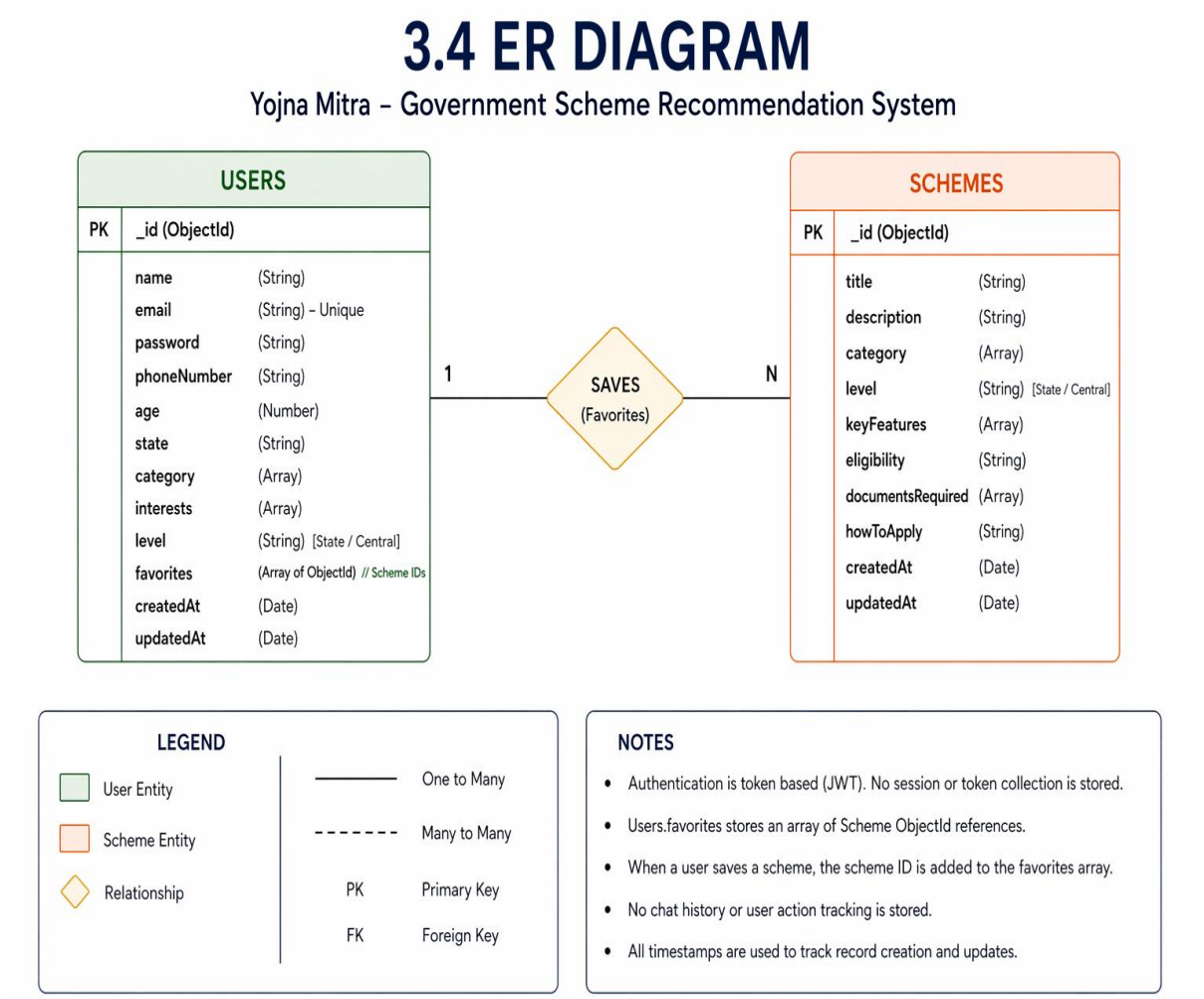
3.4 ER Diagram

The Entity-Relationship (ER) diagram represents the **data structure and relationships** used in the Yojna Mitra system. The database is designed using **MongoDB (NoSQL)**, where data is stored in collections with flexible schemas.

The system consists of two primary entities:

- **User**
- **Scheme**

The relationship between these entities is defined through a **favorites mechanism**, where users can save schemes.



3.5 Technologies Used

3.5.1 Overview

The Yojna Mitra system is developed using a combination of **modern web technologies and machine learning tools**. The system integrates a **MERN stack-based web application** with a **Python-based ML microservice**, ensuring scalability, efficiency, and real-time recommendation capabilities.

3.5.2 Frontend Technologies

React.js

- Used for building the user interface
- Component-based architecture for reusable UI elements
- Enables dynamic and responsive user experience

Axios

- Used for API communication between frontend and backend
- Handles HTTP requests and responses

3.5.3 Backend Technologies

Node.js

- Server-side JavaScript runtime
- Handles asynchronous operations efficiently

Express.js

- Web framework for building REST APIs
- Manages routing and middleware

JWT (JSON Web Token)

- Used for authentication
- Enables stateless and secure user sessions

3.5.4 Database Technology

MongoDB

- NoSQL database used to store:
    - User data
    - Scheme data
- Supports flexible schema (JSON format)
- Efficient for handling unstructured data

Mongoose (Optional)

- ODM (Object Data Modeling) library
- Simplifies interaction with MongoDB

3.5.5 Machine Learning Technologies

Python

- Used for implementing the recommendation engine
- Provides rich ecosystem for data processing

FastAPI

- Framework for building ML APIs
- High performance and fast response time
- Used to expose /recommend endpoint

Pandas

- Used for data manipulation and preprocessing
- Converts MongoDB data into structured format

Scikit-learn

- Used for implementing:
    - TF-IDF Vectorizer

- Cosine Similarity

3.5.6 Algorithms Used

1. TF-IDF (Term Frequency–Inverse Document Frequency)

- Converts textual data into numerical vectors
- Assigns importance to words based on frequency

2. Cosine Similarity

- Measures similarity between vectors
- Used to rank schemes based on relevance

3.5.7 Development Tools

- **Visual Studio Code** → Code editor
- **Postman** → API testing
- **Git & GitHub** → Version control

3.6 Software/Hardware Requirements

3.6 SOFTWARE REQUIREMENTS

Category	Requirement
Operating System	Windows / Linux / macOS
Runtime Environment	Node.js Runtime, Python 3.x
Database Server	MongoDB (Local / Atlas)
API Server	Uvicorn (for running FastAPI service)
Code Editor	Visual Studio Code
Version Control	Git, GitHub
Package Managers	npm (Node), pip (Python)

3.6 HARDWARE REQUIREMENTS

Category	Minimum Requirement	Recommended Requirement
Processor	Intel i3 or equivalent	Intel i5 or higher
RAM	4 GB	8 GB or more
Storage	10 GB free space	20 GB+ (SSD preferred)
Internet	Basic connection	High-speed stable internet

Note: The recommended configuration will provide better performance for development and deployment.

CHAPTER 4: IMPLEMENTATION & RESULT ANALYSIS

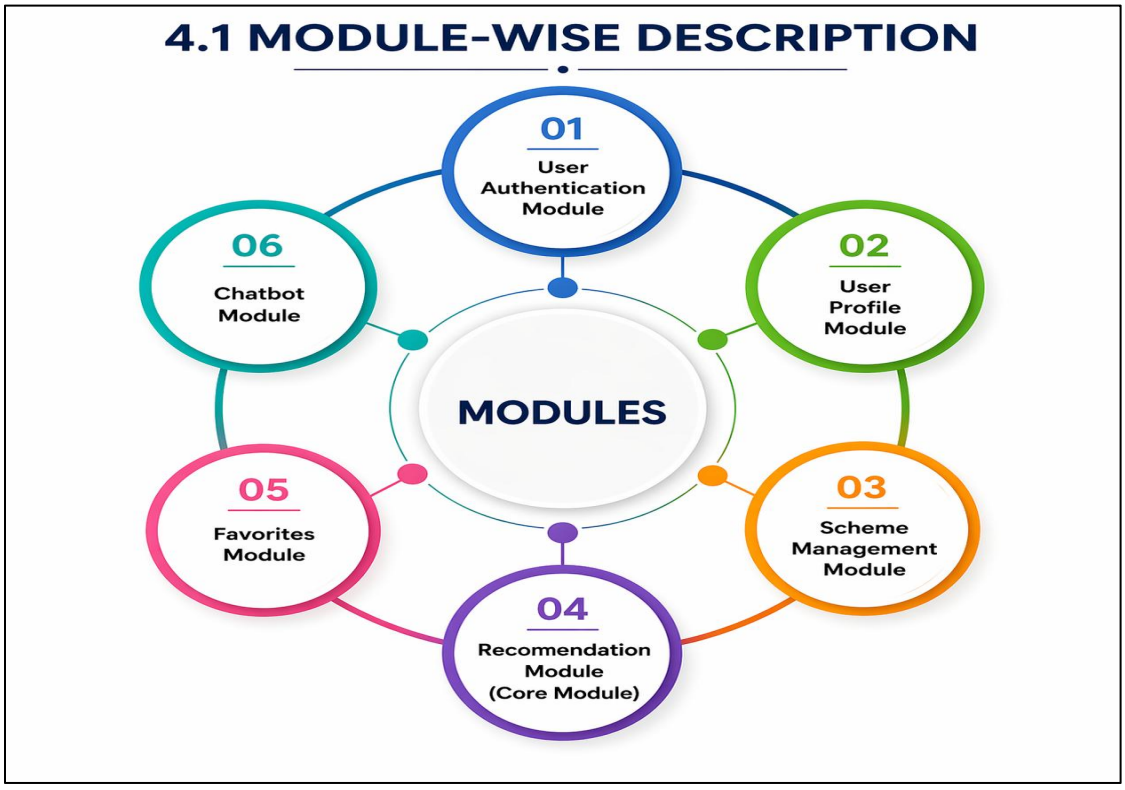
4.1 Module-wise Description

4.1.1 Overview

The Yojna Mitra system is designed using a **modular architecture**, where each module performs a specific function. This approach improves **maintainability, scalability, and clarity of implementation**. The system is divided into the following major modules:

1. User Authentication Module
2. User Profile Module
3. Scheme Management Module
4. Recommendation Module (Core ML Module)
5. Chatbot Module

Each module is explained in detail below.



4.1.2 User Authentication Module

Purpose

This module manages **user registration and login functionality** and ensures secure access to the system.

Functionality

- Allows users to create a new account (Signup)
- Authenticates existing users (Login)
- Generates and verifies JWT tokens
- Protects private routes

4.1.3 User Profile Module

Purpose

Stores and manages user-specific data used for **personalization and recommendations**.

Functionality

- View user profile
- Update profile details
- Store:
    - Category
    - Interests
    - Level
    - Age
    - State

4.1.4 Scheme Management Module

Purpose

Handles storage and retrieval of **government scheme data**.

Data Source

- Open Government Data (OGD) Platform

Functionality

- Fetch all schemes
- Display schemes in UI
- Show detailed scheme information

4.1.5 Recommendation Module (Core Module)

Purpose

This is the **core component** of the system responsible for generating personalized scheme recommendations.

Architecture

- Implemented as a **separate FastAPI microservice**

Input

- User details:
    - Category
    - Interests
    - Level
    - Age
    - Income

4.1.6 Chatbot Module

Purpose

Provides **assistance and guidance** to users regarding schemes.

Functionality

- Answer scheme-related queries

- Help users understand eligibility
- Guide navigation

4.1.7 Integration Between Modules

All modules are interconnected:

- Authentication → enables access
- Profile → feeds recommendation engine
- Scheme module → provides data
- Recommendation module → generates results
- Favorites → stores user preferences
- Chatbot → enhances usability

4.2 Coding Snippets (Detailed in Appendix)

4.2.1 Overview

This section presents the **important code snippets** used in implementing the Yojna Mitra system. The snippets highlight the **end-to-end flow** of the application, from user authentication to recommendation generation and response display.

4.2.2 Backend Initialization (Node.js)

Server Setup

```
import express from "express";
import mongoose from "mongoose";
import dotenv from "dotenv";
dotenv.config();
const app = express();
app.use(express.json());
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
```

```
    useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.log(err));
app.listen(5000, () => console.log("Server running on port 5000"));
```

4.2.3 User Authentication (JWT)

Login Controller

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
export const loginUser = async (req, res) => {
    const user = await User.findOne({ email: req.body.email });
    if (!user) return res.status(404).json({ msg: "User not found" });
    const isMatch = await bcrypt.compare(req.body.password, user.password);
    if (!isMatch) return res.status(400).json({ msg: "Invalid credentials" });
    const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
        expiresIn: "7d",
    });
    res.json({ token });
};
```

4.2.3 User Schema (MongoDB)

```
import mongoose from "mongoose";
const userSchema = new mongoose.Schema({
    name: String,
    email: { type: String, unique: true },
    password: String,
    phoneNumber: String,
    age: Number,
```

```
    state: String,
    category: [String],
    interests: [String],
    level: String,
    favorites: [{ type: mongoose.Schema.Types.ObjectId, ref: "Scheme" }],
}, { timestamps: true });
export default mongoose.model("User", userSchema);
```

4.2.5 Recommendation Controller (Backend → ML Service)

```
import axios from "axios";
export const recommendationController = async (req, res) => {
    try {
        const user = await User.findById(req.user.id);
        const response = await axios.post(process.env.ML_URL, {
            category: user.category,
            level: user.level,
        });
        res.json(response.data);
    } catch (error) {
        res.status(500).json({ msg: "ML service error" });
    }
};
```

4.2.6 FastAPI ML Service Initialization

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import Union, List
app = FastAPI()
```

```
class UserInput(BaseModel):
    category: Union[str, List[str]]
    level: str
```

4.2.7 Load Data from MongoDB (Python)

```
from pymongo import MongoClient
import pandas as pd
import os
client = MongoClient(os.getenv("MONGO_URI"))
db = client["scheme-setu"]
collection = db["schemes"]


def load_data():
    data = list(collection.find({}, {"_id": 1, "title": 1, "description": 1,
                                     "category": 1, "level": 1,
                                     "keyFeatures": 1, "eligibility": 1}))
    df = pd.DataFrame(data).fillna("")
    df["_id"] = df["_id"].astype(str)
    return df
```

4.2.8 Text Preprocessing

```
def preprocess(df):
    df["category"] = df["category"].apply(
        lambda x: " ".join(x) if isinstance(x, list) else str(x)
    )
    df["combined"] = (
        df["description"] + " " +
        df.get("keyFeatures", "").astype(str) + " " +
```

```
        df.get("eligibility", "").astype(str) + " " +
        df["category"] + " " +
        df["level"]
    )
    return df
```

4.2.9 TF-IDF Vectorization

```
from sklearn.feature_extraction.text import TfidfVectorizer

df = preprocess(load_data())

vectorizer = TfidfVectorizer(stop_words="english")

tfidf_matrix = vectorizer.fit_transform(df["combined"])
```

4.2.10 Recommendation API (Core Logic)

```
from sklearn.metrics.pairwise import cosine_similarity

@app.post("/recommend")

def recommend(user: UserInput):

    if isinstance(user.category, list):

        category_text = " ".join(user.category)

    else: category_text = user.category

    user_input = f"{category_text} {category_text} {user.level}"

    user_vector = vectorizer.transform([user_input])

    similarities = cosine_similarity(user_vector, tfidf_matrix)[0]

    top_indices = similarities.argsort()[::-1][:10]

    results = df.iloc[top_indices]
```

```
    return results.to_dict(orient="records")
```

4.2.11 Display Recommendations

```
schemes.map((scheme) => (
  <div key={scheme._id}>
    <h3>{scheme.title}</h3>
    <p>{scheme.description}</p>
  </div>
));
```

4.3 Testing Strategy

4.3.1 Overview

Testing ensures that the Yojna Mitra system functions correctly, securely, and efficiently. A combination of **unit testing, integration testing, functional testing, and performance testing** was applied to validate different components of the system.

The testing process focuses on:

- Correctness of outputs
- Reliability of APIs
- Performance of the recommendation engine
- User experience

4.3.2 Types of Testing Performed

1. Unit Testing

Purpose

To verify the correctness of individual components independently.

Modules Tested

- Authentication logic (JWT generation and validation)
- ML recommendation function
- Database queries

2. Integration Testing

Purpose

To ensure proper communication between system components.

Components Tested

- Frontend ↔ Backend API
- Backend ↔ ML Service
- Backend ↔ MongoDB

3. Functional Testing

Purpose

To verify that all features work according to requirements.

- User signup and login
- Recommendation generation
- Saving/removing favorites
- Profile update
- Chatbot interaction

All functionalities performed as expected under normal conditions.

4. Performance Testing

Purpose

To evaluate system responsiveness and efficiency.

Parameters Measured

- API response time
- ML recommendation processing time

Observations

- Recommendation results generated quickly due to:
    - TF-IDF precomputation
    - Efficient cosine similarity

5. Edge Case Testing

Purpose

To test system behavior under unusual conditions.

Test Cases

- Empty user input
- Invalid category/level
- No matching schemes
- Missing fields in database

Outcome

- System handled errors gracefully
- Default responses or empty results returned

4.3.3 Testing Tools Used

- Postman → API testing
- Browser testing → UI validation

4.3.4 Evaluation of Recommendation System

Approach

Since labeled data is not available, evaluation is based on:

- **Relevance of recommended schemes**
- **Consistency of results**
- **User satisfaction (manual validation)**

Observations

- System provides **contextually relevant schemes**
- Works well for diverse inputs
- Handles multiple categories effectively

SAMPLE TEST CASE TABLE

Test Case ID	Test Case	Input	Expected Output	Actual Output	Result
TC_01	User Login (Valid Credentials)	Email: user@example.com Password: correct123	JWT token generated successfully	JWT token generated successfully	PASS
TC_02	User Login (Invalid Password)	Email: user@example.com Password: wrong123	Error message "Invalid credentials"	Error message "Invalid credentials"	PASS
TC_03	Get Recommendations (Valid Input)	Category: Education Level: Beginner	List of relevant schemes returned	List of relevant schemes returned	PASS
TC_04	Get Recommendations (Empty Input)	Category: (empty) Level: (empty)	Validation error / default response	No validation handled, server returned 500 error	FAIL
TC_05	Add to Favorites (Valid Scheme)	Scheme ID: 665a5f8e2d9b1c3a (valid ID)	Scheme added to favorites successfully	Scheme added to favorites successfully	PASS
TC_06	Unauthorized Access (No Token)	Access protected API without JWT token	Access denied / Unauthorized error	Access denied / Unauthorized error	PASS

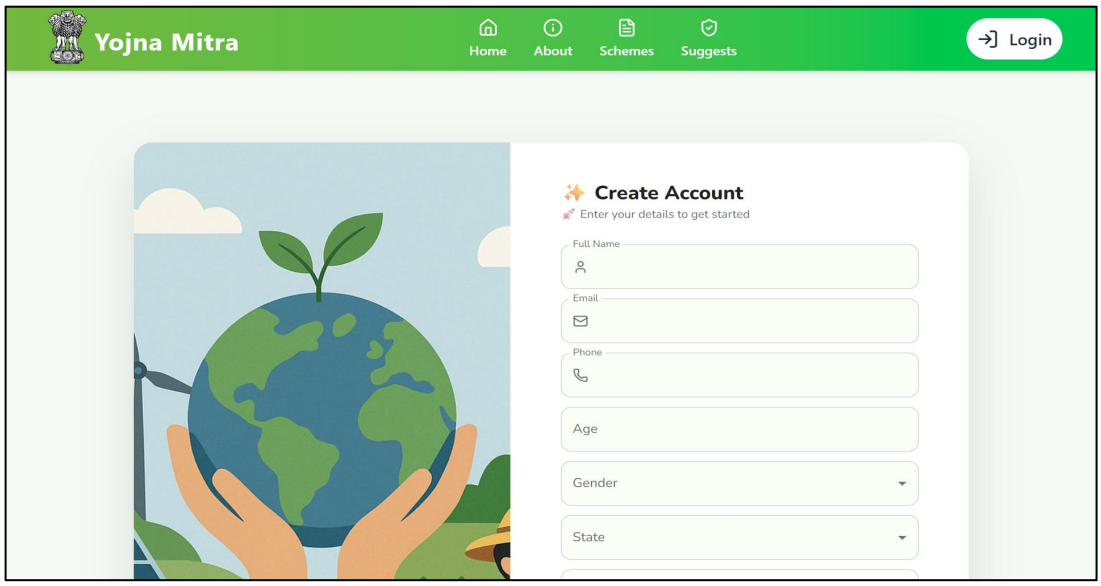
4.4 Input/Output Screenshots

4.4.1 Overview

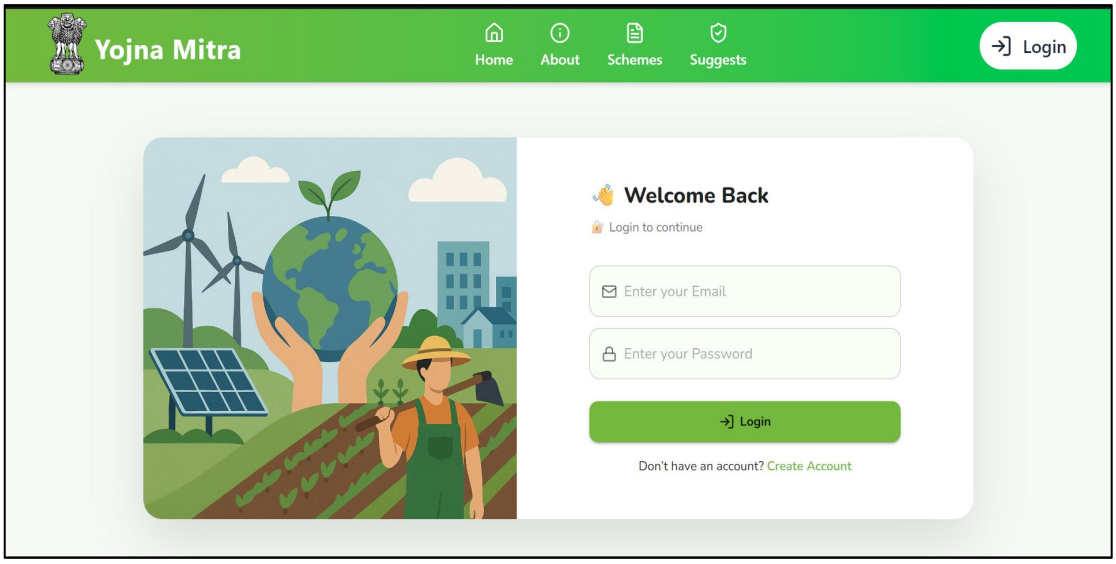
This section presents the **input and output interfaces** of the Yojna Mitra system. The screenshots demonstrate how users interact with the system and how results are displayed.

4.4.2 Input Screens

1.SignUp Page

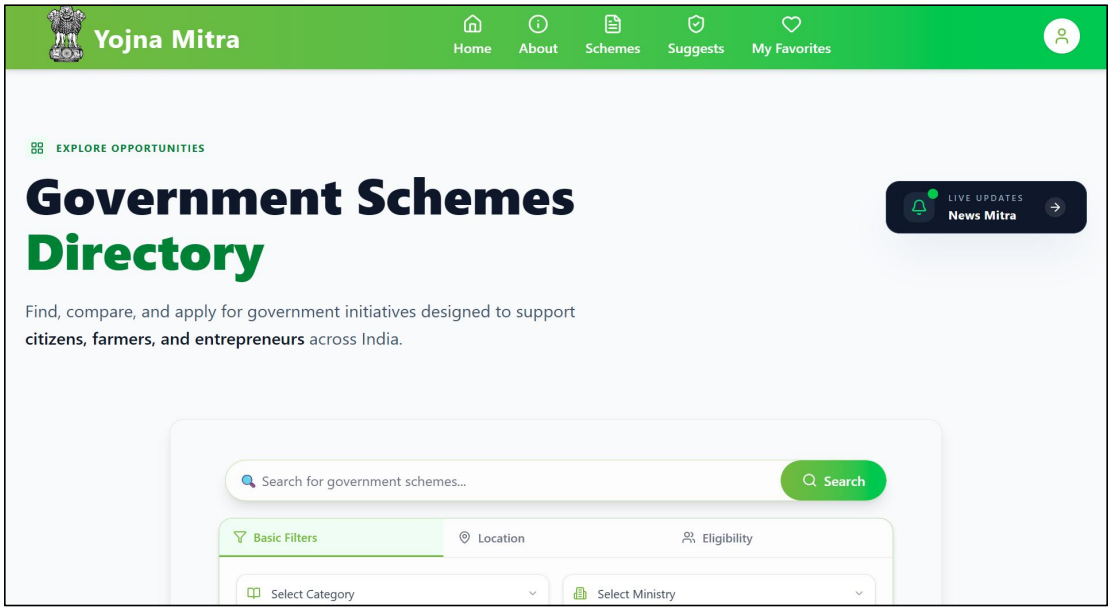


2. Login Page

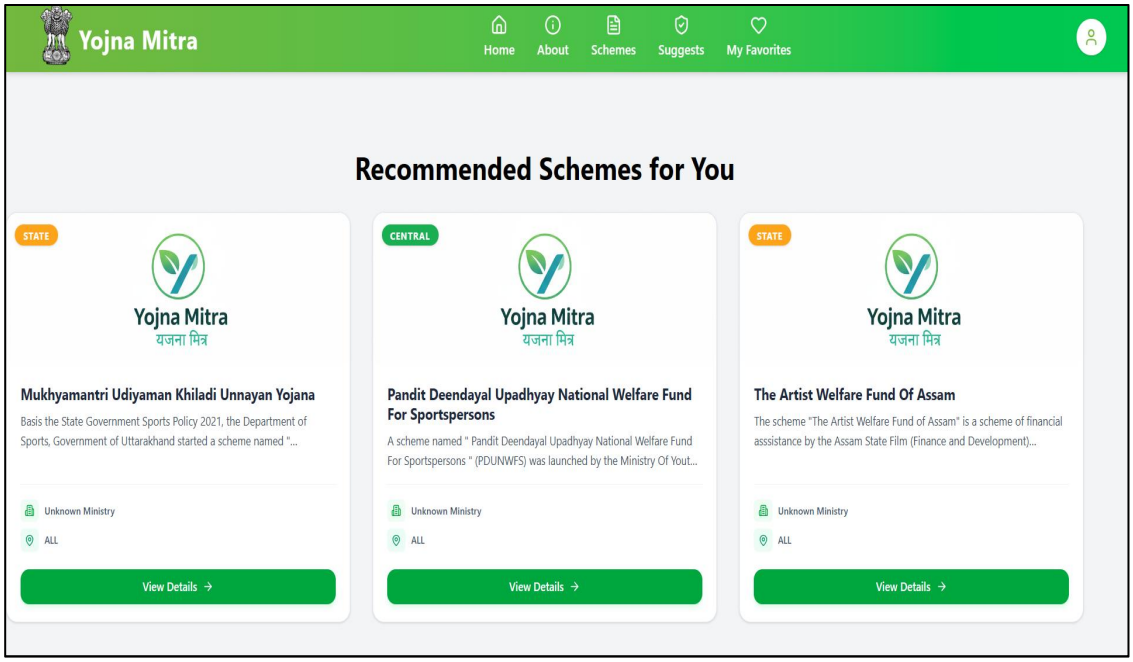


4.4.3 Output Screens

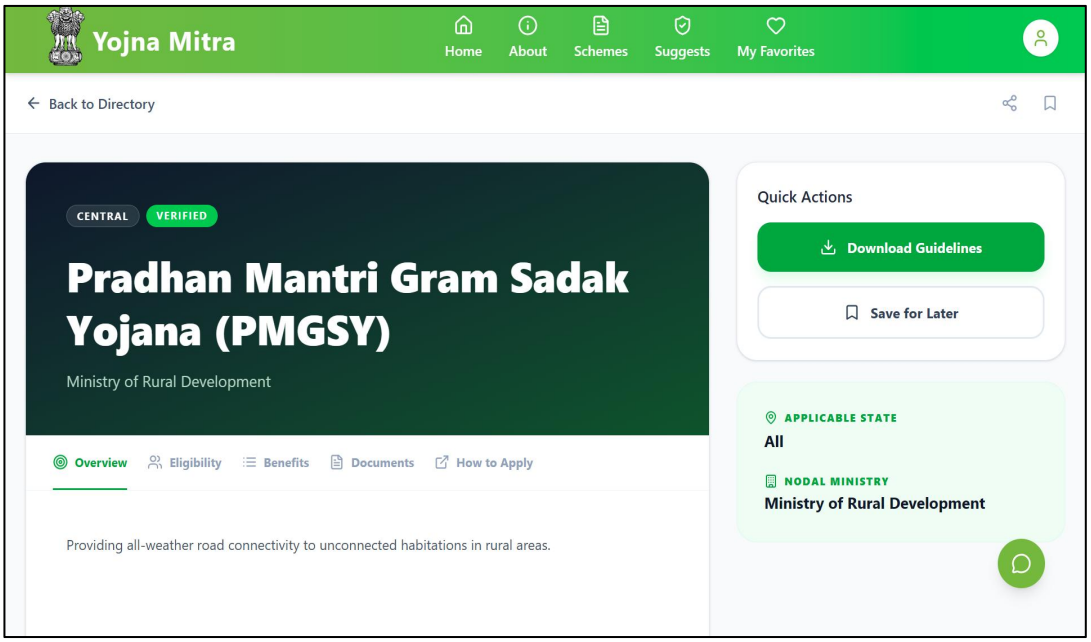
1. All Schemes Page



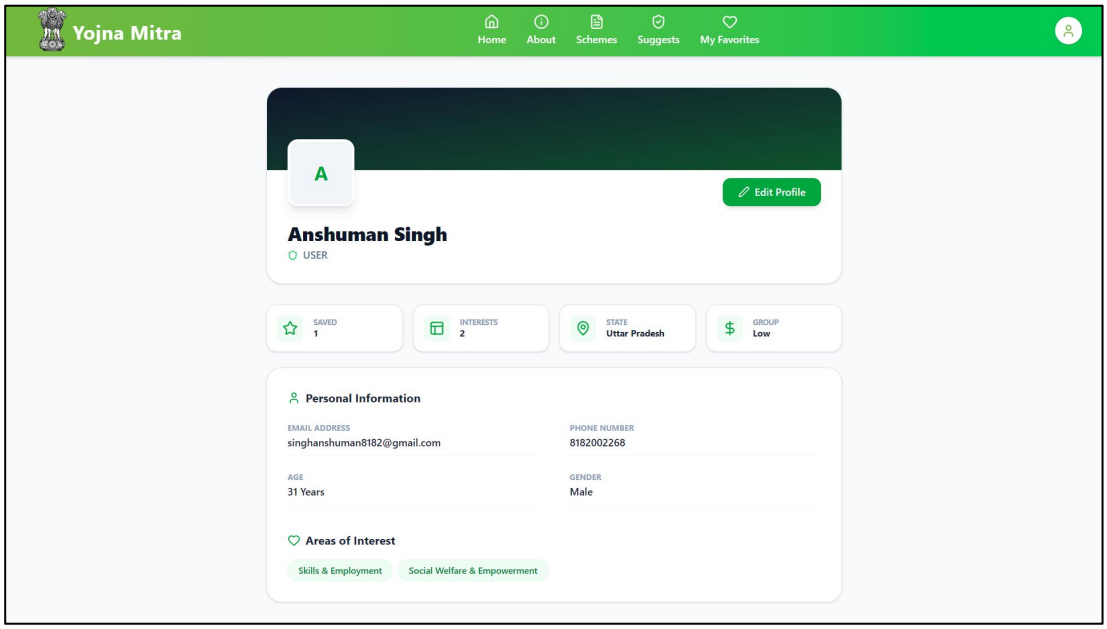
2. Recommendations Result Page



3. Scheme Details Page



4. Profile Page



4.5 Comparison with Existing Solutions

4.5.1 Overview

This section compares the proposed Yojna Mitra system with existing government scheme platforms and traditional search-based systems. The comparison highlights improvements in **personalization, usability, efficiency, and intelligence**.

4.5.2 Existing Systems

Existing platforms such as government portals primarily provide:

- Static information about schemes
- Manual search and filtering options
- Category-based browsing

Users must:

- Read multiple schemes
- Manually check eligibility
- Spend significant time finding relevant schemes

4.5.3 Proposed System

The proposed system introduces:

- **Centralized scheme access**
- **Content-based recommendation engine**
- **Personalized results based on user profile**
- **Interactive features (favorites, chatbot, profile)**

4.5.4 Key Improvements

1. Personalization of Scheme Recommendations

Existing Systems

- Provide the same list of schemes to all users
- No consideration of user-specific attributes
- Users must manually filter schemes

Proposed System

- Uses **user profile data** (category, interests, level, state, etc.)
- Applies **content-based filtering (TF-IDF + cosine similarity)**
- Generates **tailored recommendations** for each user

2.Improved Efficiency and Time Optimization

Existing Systems

- Users spend significant time:
    - Searching multiple portals
    - Reading lengthy descriptions
    - Checking eligibility manually

Proposed System

- Centralized database + ML engine
- Instant recommendation generation

4.5 Comparison with Existing Systems

Feature	Existing Systems (Government Portals / Traditional)	Proposed System (Yojna Mitra)	Improvement
Data Access	✗ Data is scattered across multiple portals	✓ Centralized platform with all schemes in one place	↑ Easy and unified access to information
Personalization	✗ No personalization; same results for all users	✓ Personalized recommendations based on user profile	↑ More relevant and user-specific results
Recommendation Engine	✗ No recommendation engine available	✓ Content-based filtering (TF-IDF + Cosine Similarity)	↑ Accurate and intelligent suggestions
Time Consumption	✗ High time consumption in searching and comparing	✓ Quick results in few seconds	↑ Reduced time significantly
Favorites Feature	✗ Not available	✓ Users can save and manage favorite schemes	↑ Helps users keep track of schemes
Chatbot Support	✗ Not available	✓ Integrated chatbot for user assistance	↑ Improved accessibility and guidance
Scalability	✗ Limited scalability and hard to extend	✓ Scalable architecture (MERN + FastAPI)	↑ Easier to scale and add new features
Real-time Results	✗ Limited or no real-time processing	✓ Real-time recommendations based on user input	↑ Instant and dynamic results

4.6 Inference from Results

4.6.1 Overview

This section presents the observations and conclusions derived from the implementation and testing of the Yojna Mitra system. The analysis evaluates the system in terms of **recommendation quality, performance efficiency, reliability, and user experience**.

4.6.2 Key Observations

1. Relevance and Accuracy of Recommendations

- The recommendation engine consistently produced **relevant scheme suggestions** based on user inputs such as category and level
- The use of **TF-IDF vectorization and cosine similarity** enabled effective matching between user requirements and scheme descriptions
- The system performed well even without labeled training data

2. Efficiency and Response Time

- Recommendations were generated **within a few seconds**
- Precomputation of TF-IDF vectors reduced processing overhead
- The system maintained stable performance under repeated requests

3. Improved User Experience

- Users were able to:
    - Quickly discover relevant schemes
    - Navigate through different modules easily
- The presence of features such as:
    - Favorites
    - Profile management
    - Chatbot enhanced usability

4. System Reliability and Stability

- All core modules functioned correctly:
    - Authentication system
    - Recommendation engine
    - Database operations
- Error handling ensured that invalid inputs did not crash the system

5. Effective Use of Unstructured Data

- The system successfully utilized textual fields such as:
    - Description
    - Benefits
    - Eligibility
- These fields were transformed into meaningful numerical representations using TF-IDF

4.6.3 Strengths of the System

- Provides **personalized recommendations**
- Centralized platform for scheme access
- Scalable architecture (MERN + FastAPI)
- Real-time processing capability
- Efficient handling of large textual datasets

4.6.4 Overall Analysis

The system demonstrates that a **content-based recommendation approach is effective for government scheme discovery**, especially when dealing with unstructured textual data. It significantly improves efficiency compared to manual search systems and provides meaningful, user-specific results.

CHAPTER 5: CONCLUSION

5.1 Summary of Work

The Yojna Mitra system was designed and developed as a **web-based centralized platform** for accessing and recommending government schemes. The primary objective of the project was to simplify the process of discovering relevant schemes by integrating a **content-based recommendation system** with a modern web application.

The system was implemented using the **MERN stack (MongoDB, Express.js, React.js, Node.js)** along with a **Python-based FastAPI microservice** for the recommendation engine. The scheme dataset was collected from the **Open Government Data (OGD) platform**, ensuring authenticity and relevance.

The development process involved multiple stages, including:

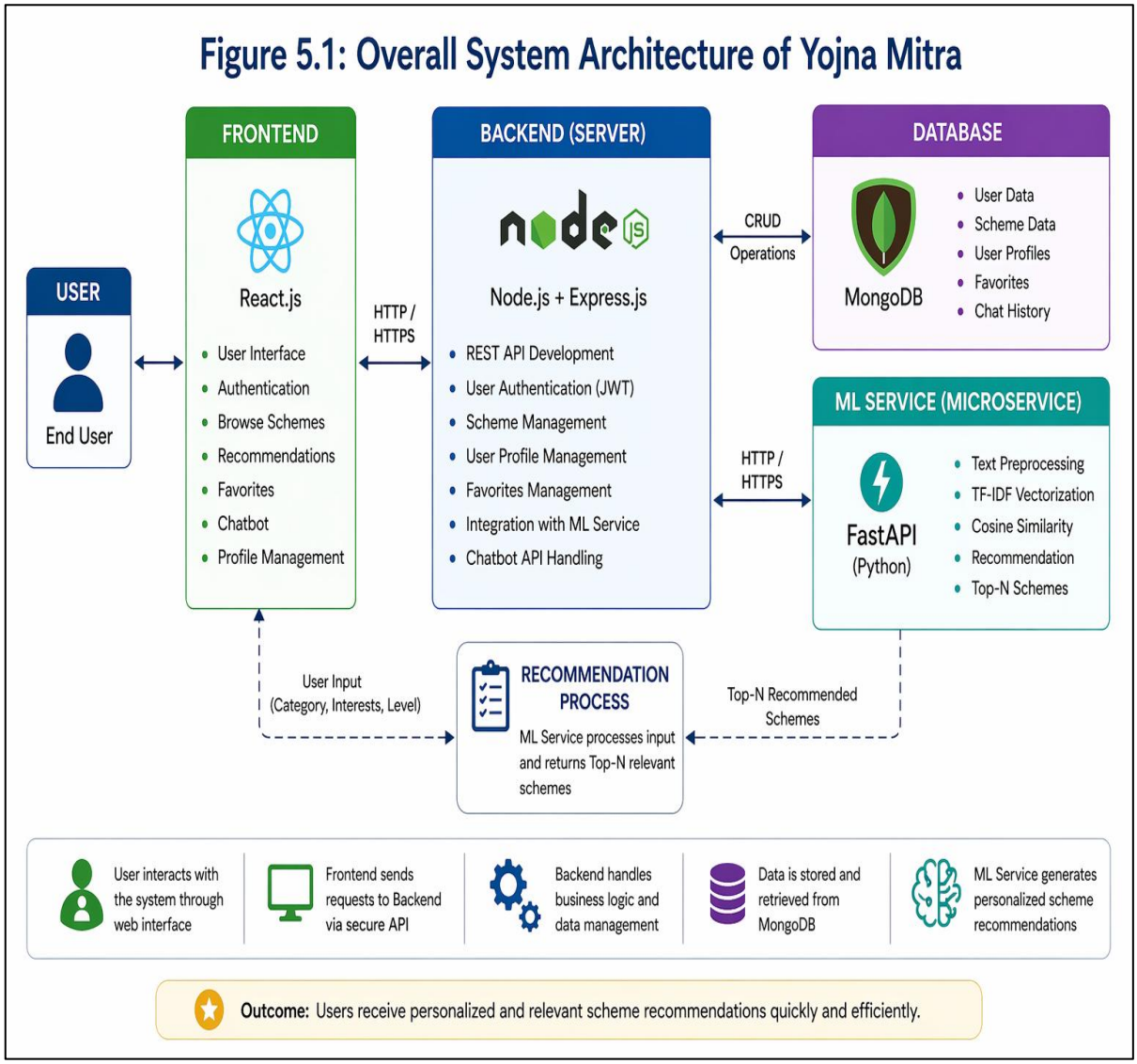
- Designing the system architecture and database schema
- Implementing user authentication using JWT
- Developing modules such as:
    - User profile management
    - Scheme browsing and detailed view
    - Recommendation system
    - Favorites management
    - Chatbot interaction

The recommendation engine was built using **TF-IDF vectorization and cosine similarity**, which enabled the system to analyze textual data such as scheme descriptions, benefits, and eligibility criteria. Based on user inputs (category, interests, and level), the system generates **personalized recommendations in real time**.

Extensive testing was performed to ensure system reliability, including unit testing, integration testing, and functional testing. The results demonstrated that the system is capable

of delivering **accurate, efficient, and user-friendly recommendations**, thereby improving the overall user experience.

Overall, the project successfully integrates **web technologies and machine learning techniques** to address the limitations of traditional government scheme portals.



5.2 Achievements

The Yojna Mitra system achieved several important outcomes during its development and implementation:

1. Successful Development of an Intelligent Recommendation System

- Designed and implemented a **content-based recommendation engine**
- Applied **TF-IDF vectorization and cosine similarity** for text analysis
- Achieved accurate matching between user requirements and scheme data
- Demonstrated effective recommendation without requiring labeled datasets

2. Transformation of Traditional Search into Automated Discovery

- Replaced manual search processes with **automated recommendation logic**
- Enabled system-driven ranking of schemes based on relevance
- Significantly reduced dependency on keyword-based filtering

3. Creation of a Centralized and Structured Data Platform

- Consolidated scheme data from multiple sources into a **single unified database**
- Standardized and organized unstructured government data
- Improved accessibility and consistency of information

4. Implementation of a Scalable Microservice-Based Architecture

- Designed a system combining:
    - MERN stack for application layer
    - FastAPI for ML microservice
- Ensured clear separation between:
    - Application logic
    - Machine learning processing

- Enabled easy scalability and independent module upgrades

5. Real-Time Personalized Recommendation Capability

- Delivered **instant recommendations** based on dynamic user input
- Ensured low latency through precomputed TF-IDF vectors
- Supported real-time interaction between frontend and ML service

6. Effective Utilization of Unstructured Textual Data

- Leveraged textual fields such as:
    - Description
    - Benefits
    - Eligibility
- Converted unstructured data into **meaningful numerical representations**
- Demonstrated practical application of NLP techniques

7.Enhancement of User Experience and Accessibility

- Designed a **responsive and intuitive user interface**
- Simplified navigation and reduced complexity for end users
- Enabled users to quickly identify relevant schemes

CHAPTER 6: FUTURE SCOPE

6.1 Limitations

While the Yojna Mitra system achieves its primary objectives, several limitations were identified that can impact performance, scalability, and recommendation quality.

1. Dependency on Textual Data Quality

- The recommendation engine relies heavily on textual fields such as:
    - Description
    - Eligibility
    - Benefits
- Inconsistent, incomplete, or noisy data reduces recommendation accuracy
- Lack of standardized schema across government datasets affects preprocessing

2. Limited Semantic Understanding (TF-IDF Constraint)

- TF-IDF is a **statistical approach**, not semantic
- Cannot capture:
    - Context
    - Synonyms
    - Intent
- Example:
    - "Financial aid" vs "monetary support" may not match effectively

3. Absence of User Feedback Loop

- System does not:
    - Learn from user clicks
    - Adapt based on preferences
- No reinforcement or iterative improvement in recommendations

4. Cold Start Problem

- New users without sufficient profile data:
    - Receive generic recommendations
- System lacks behavioral data to refine results

5. Limited Personalization Dimensions

- Current personalization is based on:
    - Category
    - Interests
    - Level
- Does not include:
    - Occupation
    - Demographic patterns
    - Behavioral analytics

6. Stateless Chatbot Limitation

- Chatbot does not:
    - Maintain conversation history
    - Understand context across queries
- Limits depth of interaction

7.Mobile Application Development

- Develop Android/iOS app
- Increase accessibility and user reach

6.2 Recommendations for Enhancement

To extend the capabilities of the Yojna Mitra system and address existing limitations, several advanced enhancements are proposed. These improvements focus on intelligence, scalability, personalization, and real-world deployment readiness.

1. Hybrid Recommendation System (Content + Collaborative Filtering)

Enhancement

- Integrate:
    - Content-based filtering (current system)
    - Collaborative filtering (user behavior-based)

Implementation Approach

- Use:
    - Matrix factorization (SVD)
    - User-item interaction matrix
- Combine scores using weighted hybrid model

Benefits

- Improves recommendation accuracy
- Solves cold-start problem
- Captures both content relevance and user behavior

2. Adoption of Advanced NLP Models (Semantic Understanding)

Enhancement

- Replace TF-IDF with:
    - BERT / Sentence Transformers

Implementation Approach

- Convert scheme text into **dense embeddings**
- Use cosine similarity on embeddings

- Store embeddings for faster retrieval

Benefits

- Captures context and semantics
- Handles synonyms and intent
- Significantly improves recommendation quality

3. Feedback-Driven Adaptive Learning System

Enhancement

- Introduce user feedback loop

Implementation Approach

- Collect:
    - Click-through rate (CTR)
    - Favorites
    - Time spent on schemes
- Update ranking using:
    - Reinforcement learning / ranking models

Benefits

- System continuously improves
- Personalized recommendations evolve over time

4. Behavioral Analytics and User Profiling

Enhancement

- Track user activity for deeper personalization

Implementation Approach

- Maintain logs for:

    - Search queries
    - Click patterns

- Build dynamic user profiles

Benefits

- Context-aware recommendations
- Better understanding of user intent

5. Mobile Application Development

Enhancement

- Extend platform accessibility

Implementation Approach

- Develop:
    - Android (React Native / Flutter)
- Integrate with existing backend APIs

Benefits

- Wider reach
- Improved user engagement

REFERENCES

Books

1. Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer.
2. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
3. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.

Research Papers

1. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5)
2. Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. *Proceedings of the First Instructional Conference on Machine Learning*.
3. Ricci, F., Rokach, L., & Shapira, B. (2011). Introduction to recommender systems handbook. *Springer*.

Web Resources / Documentation

1. Scikit-learn Developers. (2024). *TfidfVectorizer Documentation*. https://scikitlearn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html
2. FastAPI Documentation. (2024). https://fastapi.tiangolo.com
3. MongoDB Inc. (2024). *MongoDB Documentation*. https://www.mongodb.com/docs
4. Node.js Foundation. (2024). *Node.js Documentation*. https://nodejs.org
5. Open Government Data Platform India. (2024). https://data.gov.in

APPENDICES

A: Source Code (Summary or Sample pages)

A.1 Overview

This appendix presents **additional supporting source code** that complements the implementation discussed in Chapter 4.This appendix presents selected **complete code modules** from the Yojna Mitra system to demonstrate the implementation of backend services, authentication, routing, and the machine learning recommendation engine.

A.2 Backend Server (Complete Setup – Node.js)

```
import express from "express";
import mongoose from "mongoose";
import dotenv from "dotenv";
import cors from "cors";
import routes from "./routes/index.js";

dotenv.config();

const app = express();

// Middleware
app.use(express.json());
app.use(cors());

// Routes
app.use("/api", routes);

// MongoDB Connection
mongoose.connect(process.env.MONGO_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => console.log("MongoDB Connected"))
.catch(err => console.error(err));

// Error Handling Middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ msg: "Internal Server Error" });
});

// Server Start
const PORT = process.env.PORT || 5000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

A.3 Authentication + Middleware

```
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import User from "../models/User.js";

// Login Controller
export const loginUser = async (req, res) => {
  try {
    const { email, password } = req.body;

    const user = await User.findOne({ email });
    if (!user) return res.status(404).json({ msg: "User not found" });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.status(400).json({ msg: "Invalid credentials" });

    const token = jwt.sign(
      { id: user._id },
      process.env.JWT_SECRET,
      { expiresIn: "7d" }
    );

    res.json({ token });
  } catch (error) {
    res.status(500).json({ msg: "Login error" });
  }
};

// Auth Middleware
export const authMiddleware = (req, res, next) => {
  const token = req.headers.authorization?.split(" ")[1];

  if (!token) return res.status(401).json({ msg: "Unauthorized" });

  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    req.user = decoded;
    next();
  } catch {
    res.status(401).json({ msg: "Invalid token" });
  }
};
```

A.4 Scheme Schema

```
import mongoose from "mongoose";

const schemeSchema = new mongoose.Schema({
  title: String,
  description: String,
  category: [String],
  level: String,
  benefits: String,
  eligibility: String
});

export default mongoose.model("Scheme", schemeSchema);
```

A.5 Recommendation Controller

```
export const recommendationModelController = async (req, res) => {
    try {
        const page = parseInt(req.query.page) || 1;
        const limit = parseInt(req.query.limit) || 9;

        const userId = req.user._id;

        const userProfile = await User.findById(userId);

        if (!userProfile) {
            return res.status(404).json({
                success: false,
                message: "User not found"
            });
        }
        const { interests } = userProfile;

        // Call ML API
        const response = await axios.post("http://127.0.0.1:8000/recommend", {
            interests,

        });

        let schemes = response.data;

        // ---------------- Pagination (IMPORTANT)
        const startIndex = (page - 1) * limit;
        const paginatedSchemes = schemes.slice(startIndex, startIndex + limit);

        return res.status(200).json({
            success: true,
            total: schemes.length,
            page,
            limit,
            data: paginatedSchemes
        });

    } catch (error) {
        console.error("Recommendation Error:", error);

        return res.status(500).json({
            success: false,
            message: "Failed to fetch recommendations"
        });
    }
};
```

A.6 FastAPI ML Service

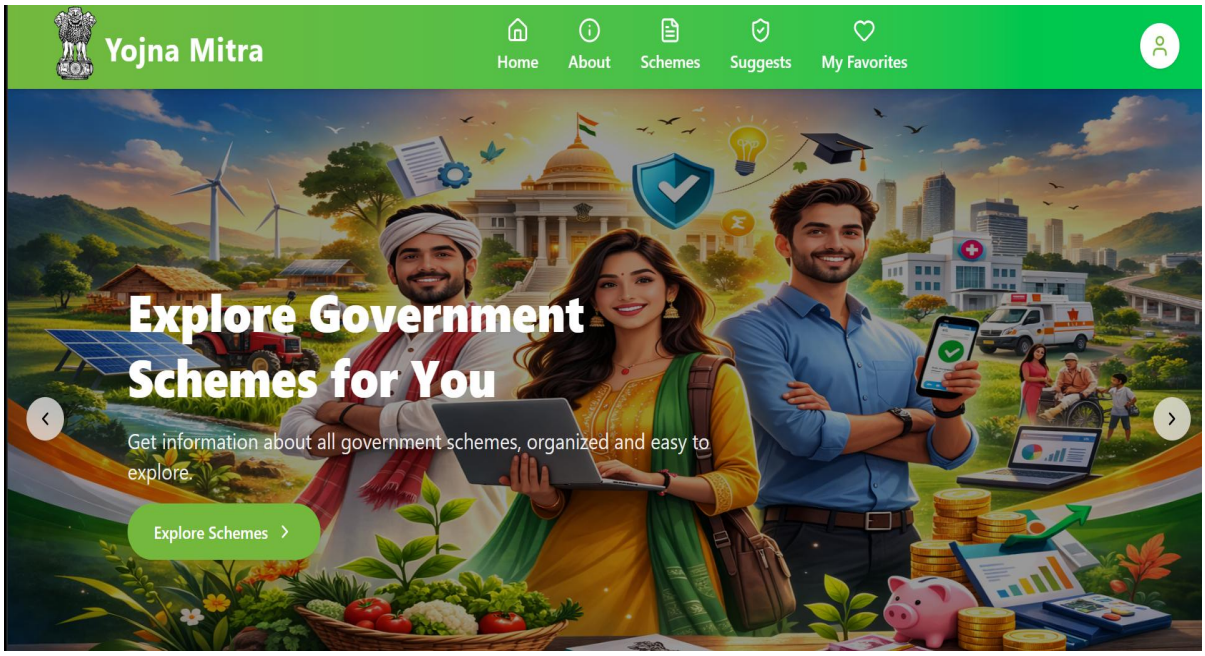
```python
import os
import time
import pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel
from pymongo import MongoClient
from dotenv import load_dotenv
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from typing import Union, List
from typing import Optional

# ---------------- LOAD ENV
load_dotenv()

MONGO_URI = os.getenv("MONGODB_URL")
DB_NAME = "test"
COLLECTION_NAME = "schemes"

# ---------------- APP INIT
app = FastAPI()

# ---------------- DB CONNECTION
client = MongoClient(MONGO_URI)
db = client[DB_NAME]
collection = db[COLLECTION_NAME]

# ---------------- LOAD & TRANSFORM DATA
def load_data():
    data = list(collection.find({}))

    if not data:
        raise Exception("No data found in MongoDB")

    df = pd.DataFrame(data).fillna("")

    # Convert ObjectId → string
    df["_id"] = df["_id"].astype(str)

    # category (array → string)
    df["category"] = df["category"].apply(
        lambda x: " ".join(x) if isinstance(x, list) else str(x)
    )

    df["documents"] = df.get("documentsRequired", "")
    df["application"] = df.get("howToApply", "")
    df["benefits"] = df.get("keyFeatures", "")
    df["eligibility"] = df.get("eligibility", "")
```

A.7 Sample API Response

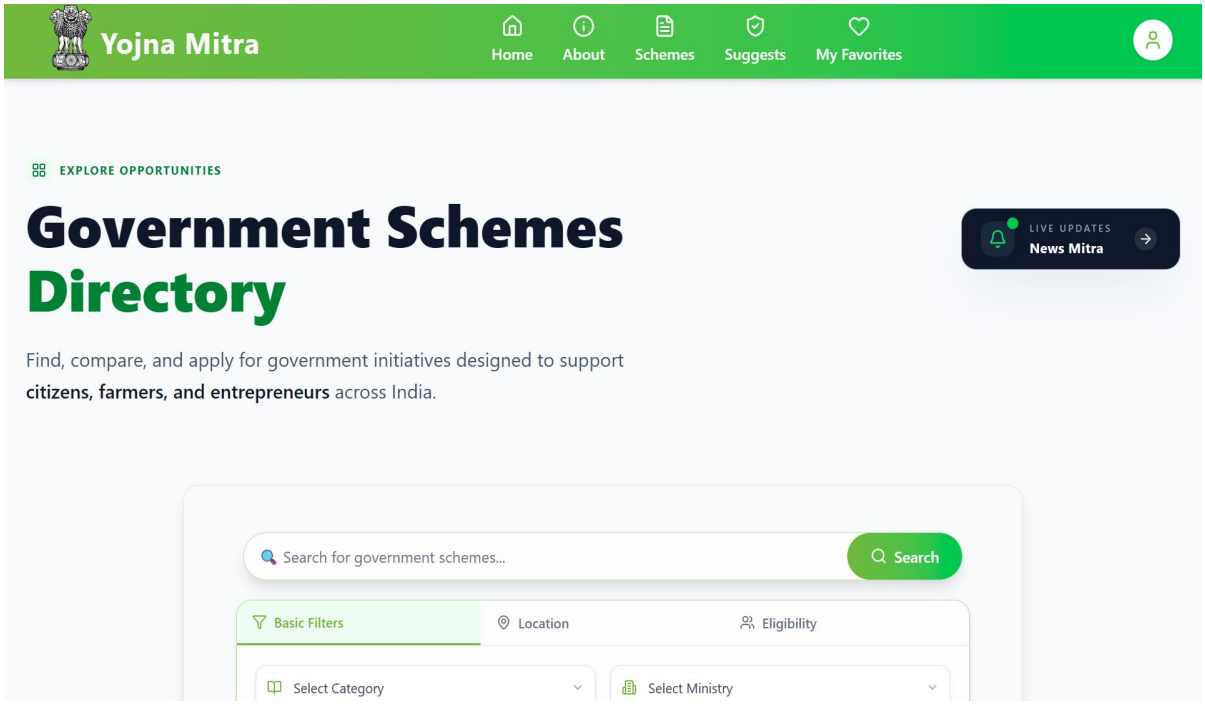

```
[
  {
    "title": "Startup India Scheme",
    "category": ["Entrepreneurship"],
    "level": "National",
    "description": "Supports startups with funding and mentorship"
  }
]
```

B: Screenshots

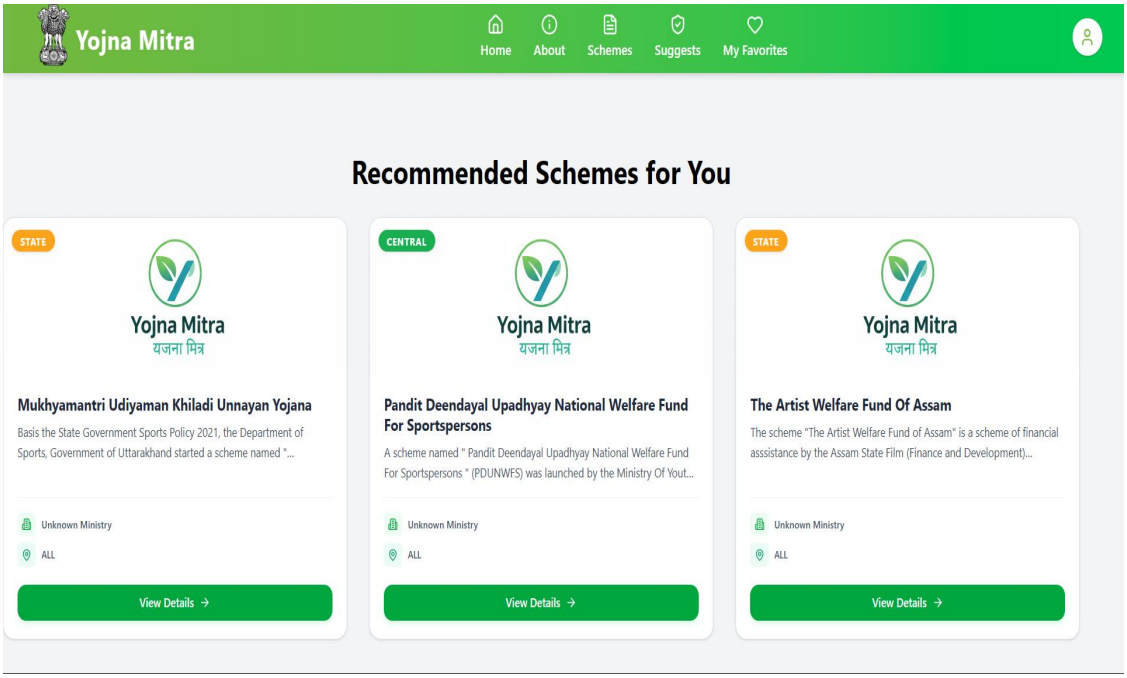
B.1: Home Page



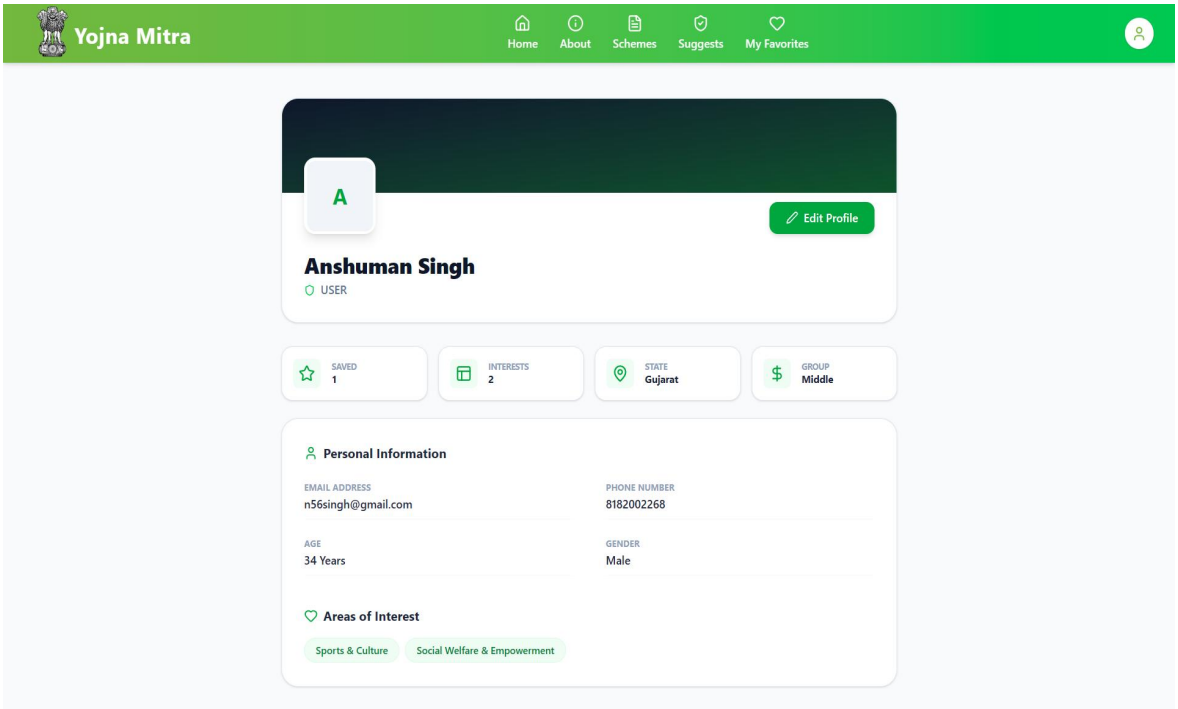
B.2: Schemes Page



B.3: Recommendations Page



B.4: Profile Page



C: Glossary/ Abbreviations

C.1 Overview

This section defines key technical terms and abbreviations used in the Yojna Mitra system, particularly those related to the recommendation engine and web application architecture.

C.2 Abbreviations

Abbreviation	Full Form
ML	Machine Learning
TF-IDF	Term Frequency – Inverse Document Frequency
API	Application Programming Interface
JWT	JSON Web Token
UI	User Interface
DB	Database
NLP	Natural Language Processing
MERN	MongoDB, Express.js, React.js, Node.js
HTTP	HyperText Transfer Protocol
REST	Representational State Transfer

C.3 Glossary of Terms

Term	Description
Recommendation	A system that suggests relevant items (schemes) to users based

Term	Description
System	on their preferences
Content-Based Filtering	A recommendation technique that uses item features to suggest similar items
TF-IDF	A numerical technique to evaluate the importance of words in a document
Cosine Similarity	A measure used to determine similarity between two vectors
Vectorization	Process of converting text into numerical form for machine learning
Microservice	A small, independent service responsible for a specific function (e.g., ML service)
Schema	Structure defining how data is stored in a database
Endpoint	API route used to send or receive data
Token-Based Authentication	Security mechanism using tokens (JWT) instead of sessions
Unstructured Data	Data without a predefined format, such as text descriptions